

UNIVERSIDAD DEL NORTE
FACULTAD DE INGENIERIA
CARRERA DE INGENIERÍA EN INFORMÁTICA



Asignatura: Programación IV

Docente: Prof. Marcos Arrieta Moreira

“Plataforma Web SantaMed”

Informe – Segundo Avance:

- ✓ Reporte de Sprints / Flujo Kanban acumulado.
- ✓ Módulo CRUD principal funcional con reglas de negocio.
- ✓ Manejo de sesiones centralizadas (Redis/BD).
- ✓ Despliegue inicial del Stack en Portainer.

Integrantes:

Nº	Nombres y Apellidos	Cédula de Identidad	Rol en el Equipo
1	Jazmín Del Cielo Betharran Zorrilla Maciel	5.437.546	Scrum Master / Project Manager
2	Alcides Fernando Monges Chávez	5.419.565	Backend & Software Architect Lead
3	Christian Gabriel Gómez Fariña	3.433.430	DevOps
4	Hugo Daniel Vera Jara	4.630.975	Cloud Engineer
5	Aldo Lorenzo Rojas Díaz	5.771.607	Frontend
6	Lucia Belén Meza Iturbe	5.814.127	QA Engineer

Asunción – Paraguay

Fecha de entrega: 02/10/2026

1. Reporte de Sprints / Kanban acumulado

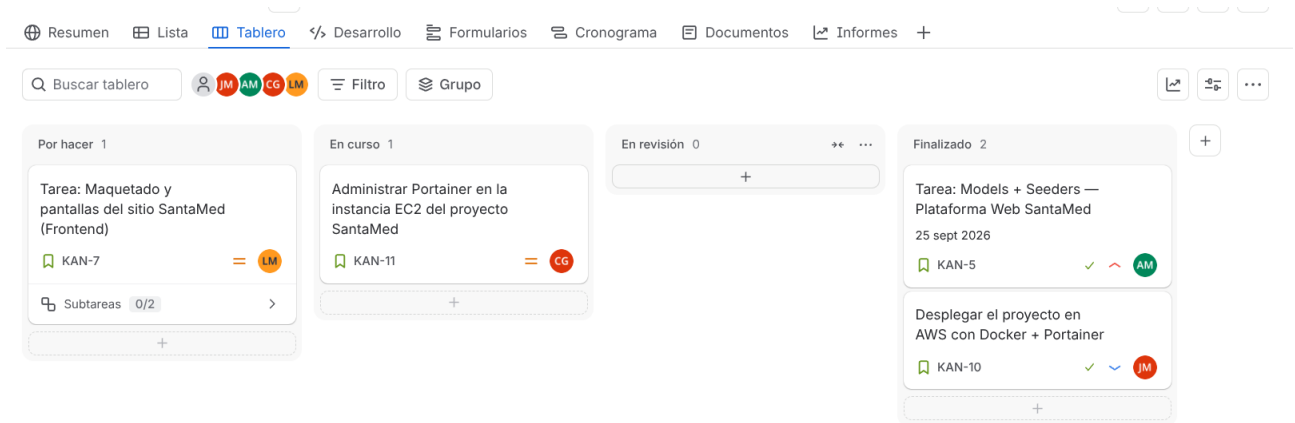
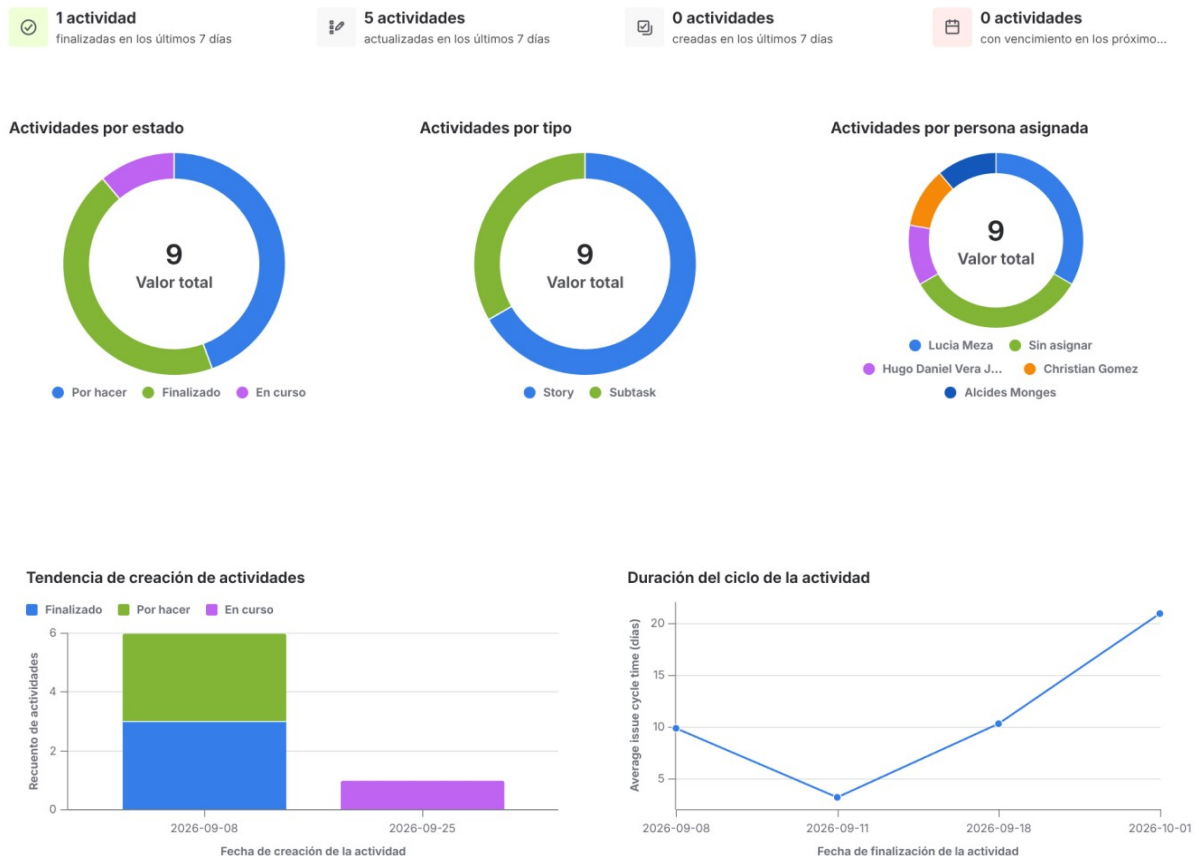


Figura 1: Tablero Kanban del proyecto en Jira. Cada tarjeta es una tarea asignada a un integrante según su rol, y avanza por las columnas a medida que se trabaja. Así el equipo ve en todo momento qué está pendiente, qué está en curso y qué ya se terminó.



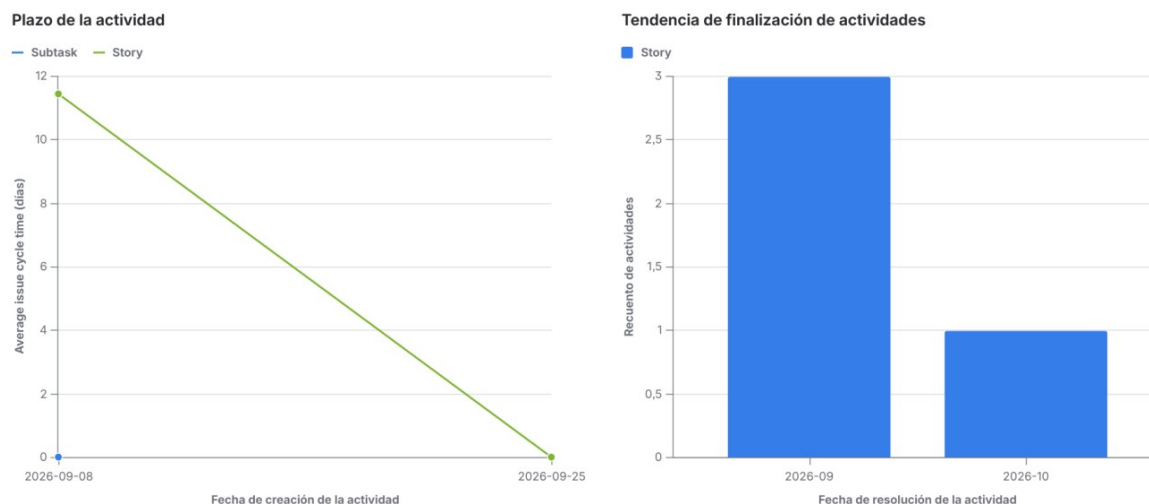


Figura 2: Resumen del proyecto generado por Jira. Muestra cuántas tareas hay en cada estado, cómo se reparte el trabajo entre los integrantes y cuántos días tarda en promedio una tarea en completarse. Lo usamos para detectar tareas trabadas y equilibrar la carga del equipo.

2. CRUD con reglas de negocio

```

{
  "data": [
    {
      "id": 12,
      "nombre": "Dr. César Martínez",
      "rango_edad_atencion": null,
      "especialidad": {
        "id": 9,
        "nombre": "Neumología"
      },
      "estudios": [
        {
          "id": 40,
          "nombre": "Espirometría con prueba broncodilatadora",
          "categoria": null
        },
        {
          "id": 41,
          "nombre": "Espirometría basal",
          "categoria": null
        }
      ],
      "horarios": [
        {
          "id": 22,
          "dia_semana": "jueves",
          "hora_inicio": "16:30",
          "hora_fin": "18:30"
        },
        {
          "id": 23,
          "dia_semana": "lunes",
          "hora_inicio": "14:00",
          "hora_fin": "17:00"
        },
        {
          "id": 24,
          "dia_semana": "martes",
          "hora_inicio": "10:45",
          "hora_fin": "12:30"
        },
        {
          "id": 25,
          "dia_semana": "viernes",
          "hora_inicio": "10:00",
          "hora_fin": "12:00"
        }
      ]
    }
  ]
}

```

Figura 3: Respuesta de la API al consultar el listado de profesionales, desplegada en el servidor de AWS. Cada profesional viene con su especialidad, los estudios que realiza y sus horarios de atención, que es la información que después va a mostrar el sitio web de la clínica.

```

app > Http > Requests > ProfessionalRequest.php > ...
1  <?php
2  |
3  namespace App\Http\Requests;
4
5  use Illuminate\Foundation\Http\FormRequest;
6  use Illuminate\Validation\Rule;
7  use Illuminate\Validation\Validator;
8
9  /**
10   * Validación y reglas de negocio para alta y edición de profesionales.
11   *
12   * Reglas de negocio:
13   * - El nombre del profesional es único.
14   * - La especialidad debe existir.
15   * - Los estudios asignados deben existir y no repetirse.
16   * - Cada horario: día válido (lunes a sábado), formato HH:MM y hora_fin posterior a hora_inicio.
17   * - Los horarios de un mismo profesional no pueden superponerse en el mismo día.
18   */
19  class ProfessionalRequest extends FormRequest
20  {
21      // ...

```

Figura 4: Reglas de negocio del CRUD. Antes de guardar, la API valida que el nombre no esté repetido, que la especialidad y los estudios existan, que solo se carguen días de lunes a sábado y que un profesional no tenga dos horarios superpuestos el mismo día. Si algo no cumple, devuelve un error claro y no guarda nada.

```

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.19s

PASS Tests\Feature\ProfesionalApiTest
✓ crea un profesional con estudios y horarios 0.13s
✓ lista y filtra profesionales 0.02s
✓ muestra un profesional 0.01s
✓ actualiza y reemplaza horarios 0.01s
✓ patch parcial solo cambia lo enviado 0.01s
✓ elimina un profesional y sus relaciones 0.01s
✓ campos obligatorios 0.01s
✓ no permite nombre duplicado 0.01s
✓ especialidad y estudios deben existir 0.01s
✓ hora fin debe ser posterior a inicio 0.01s
✓ día inválido
✓ no permite horarios superpuestos el mismo día
✓ catalogos

Tests: 15 passed (61 assertions)
Duration: 0.60s

```

Figura 5: Ejecución de las pruebas automatizadas del CRUD: 15 tests aprobados.

3. Sesiones centralizadas en Redis

```

ubuntu@ip-172-31-31-18:/var/www/SantaMedAca$ grep -E 'SESSION_DRIVER|CACHE_STORE|REDIS_HOST|REDIS_CLIENT' .env
SESSION_DRIVER=redis
CACHE_STORE=redis
REDIS_CLIENT=predis
REDIS_HOST=redis
ubuntu@ip-172-31-31-18:/var/www/SantaMedAca$

```

Figura 6: Configuración de la aplicación para guardar las sesiones y la caché en Redis en lugar de archivos locales. Esto prepara el sistema para trabajar con varios servidores detrás del balanceador HAProxy sin que los usuarios pierdan su sesión.

```

ubuntu@ip-172-31-31-18:/var/www/SantaMedApp$ docker compose exec redis redis-cli keys '*'
1) "laravel-database-laravel-cache-CuhnVM1PKmHZN6aexeAzn8wUtyltzMXsirDCnls"
2) "laravel-database-laravel-cache-RWKF1vb3KtLrPFS3TfnRchiAV640qVYEKs5NfrZV"
3) "laravel-database-laravel-cache-3eVyr2aa2ImTqAwhms8cblTC277hoG0fpBcghvHZ"
4) "laravel-database-laravel-cache-QoZJs3pI1kHbTHVBkLUYY46C16RK90wFhobdL9J"
5) "laravel-database-laravel-cache-yUsqeRbZPX5x3t8EPLqphASPUT5HcHrHzodzR67z"
6) "laravel-database-laravel-cache-SQ1dVg6tHrACZk1sk6PF2HvF55AU8q2t8eWhy1Q1"
7) "laravel-database-laravel-cache-udvRTVwCAf0x3NQvyHe82hzg2LrWzdJwvfrCX6fy"
ubuntu@ip-172-31-31-18:/var/www/SantaMedApp$

```

Figura 7: Claves almacenadas dentro del contenedor Redis. Confirman que las sesiones de los usuarios efectivamente se están guardando ahí y no en el disco del servidor.

4. Despliegue en AWS + Portainer

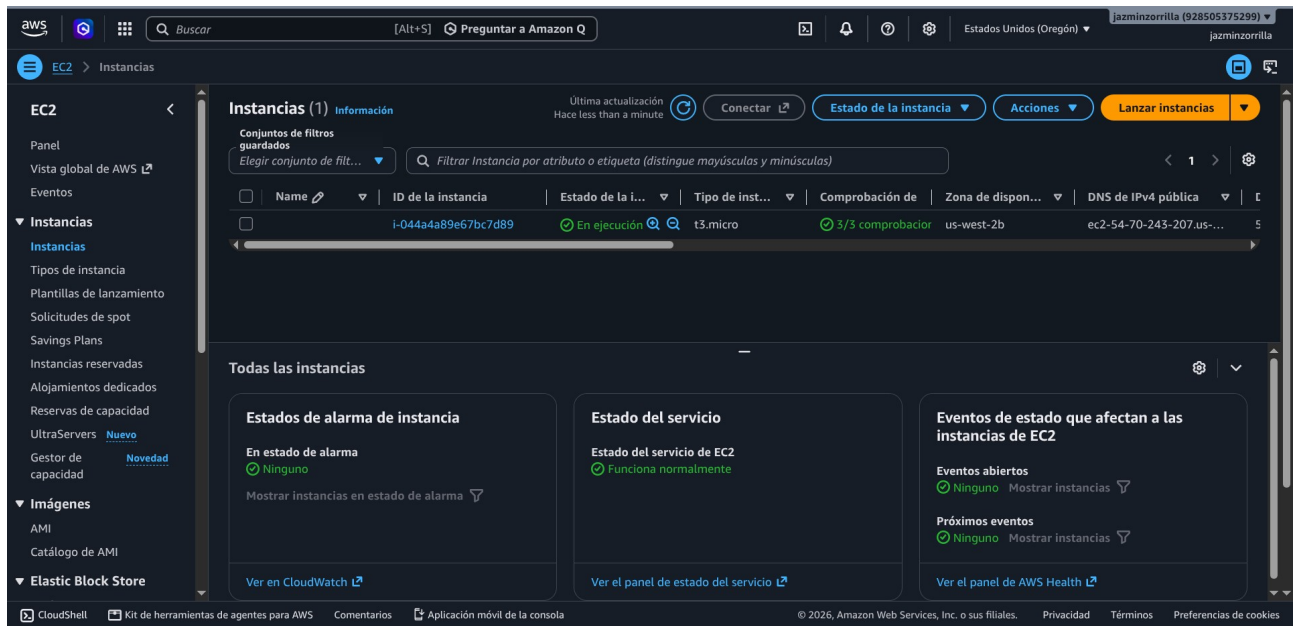


Figura 8: Instancia EC2 de AWS donde está desplegado el proyecto. Es un servidor Ubuntu con Docker que corre todos los componentes del sistema y es accesible desde internet mediante su IP pública.

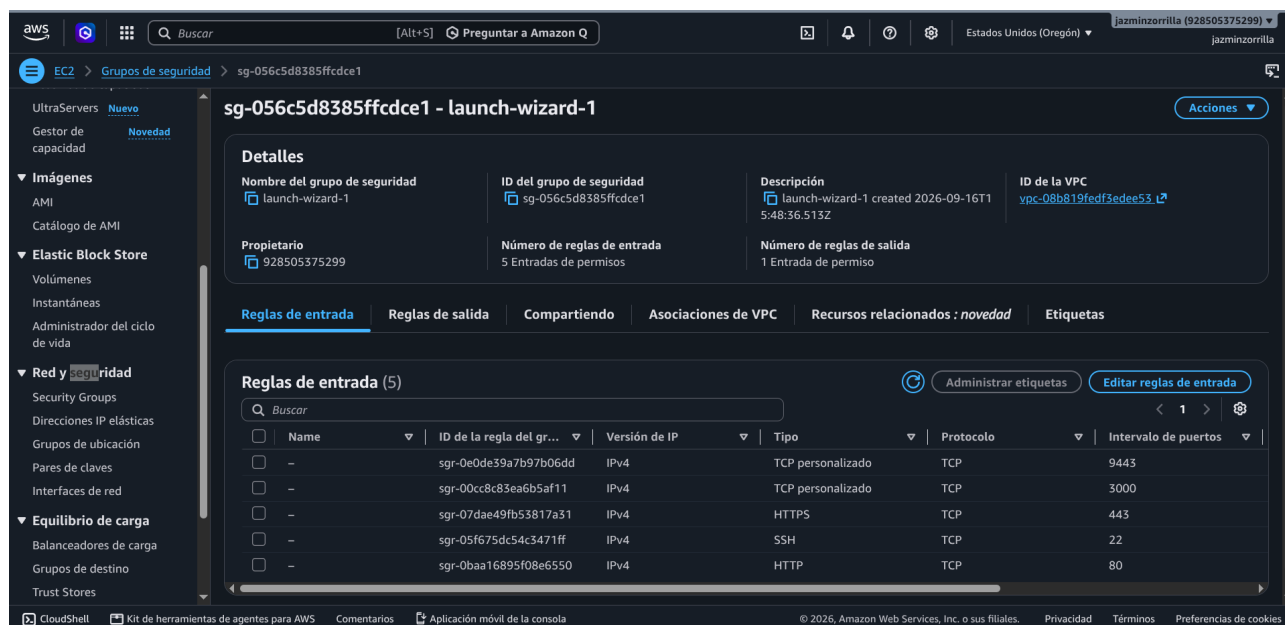
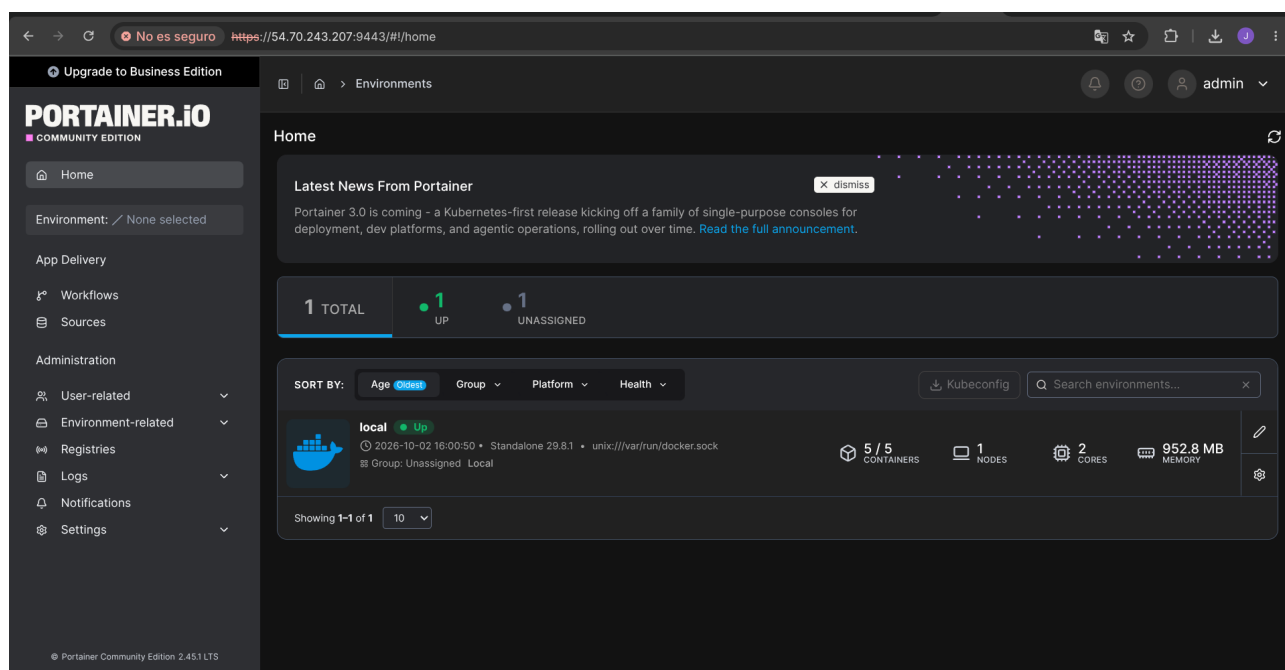


Figura 9: Reglas del firewall (Security Group) de la instancia. Solo se abren los puertos que se necesitan: la web, la API, Portainer y SSH para administración. La base de datos y Redis quedan cerrados al exterior para protegerlos.



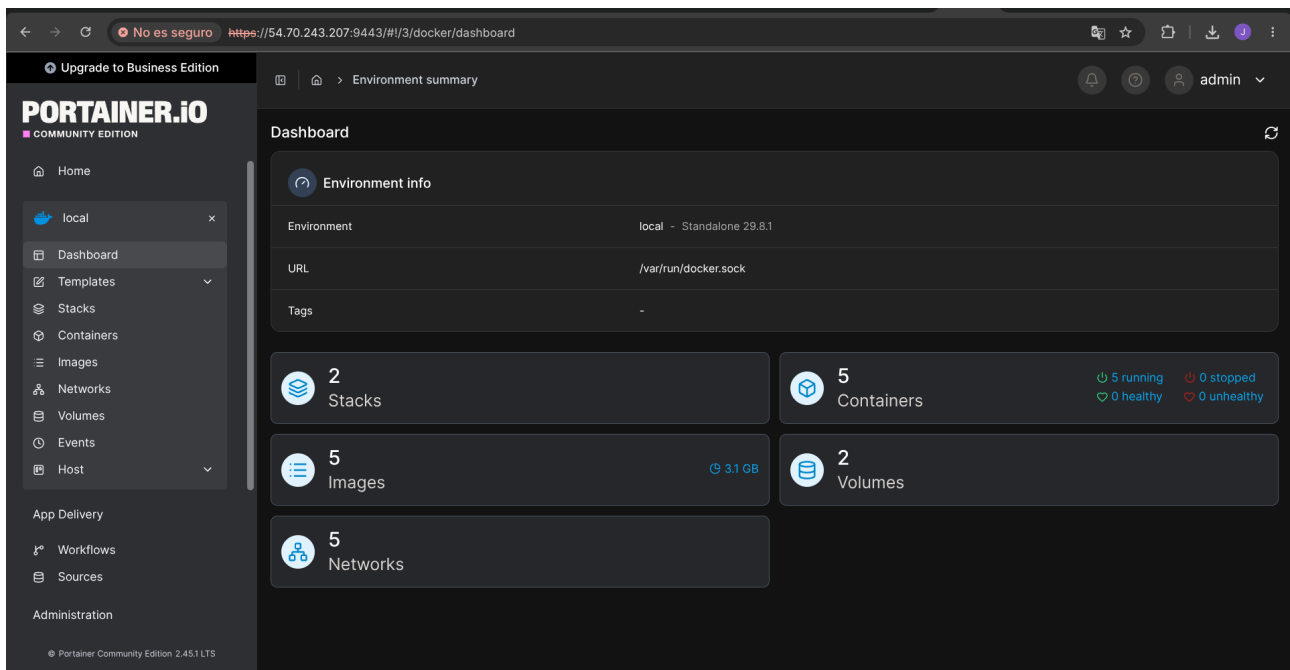


Figura 10: Panel de Portainer con los contenedores del proyecto: backend, frontend, base de datos y Redis. Permite al equipo ver el estado de cada servicio, revisar logs y reiniciarlos desde el navegador, sin entrar por consola.

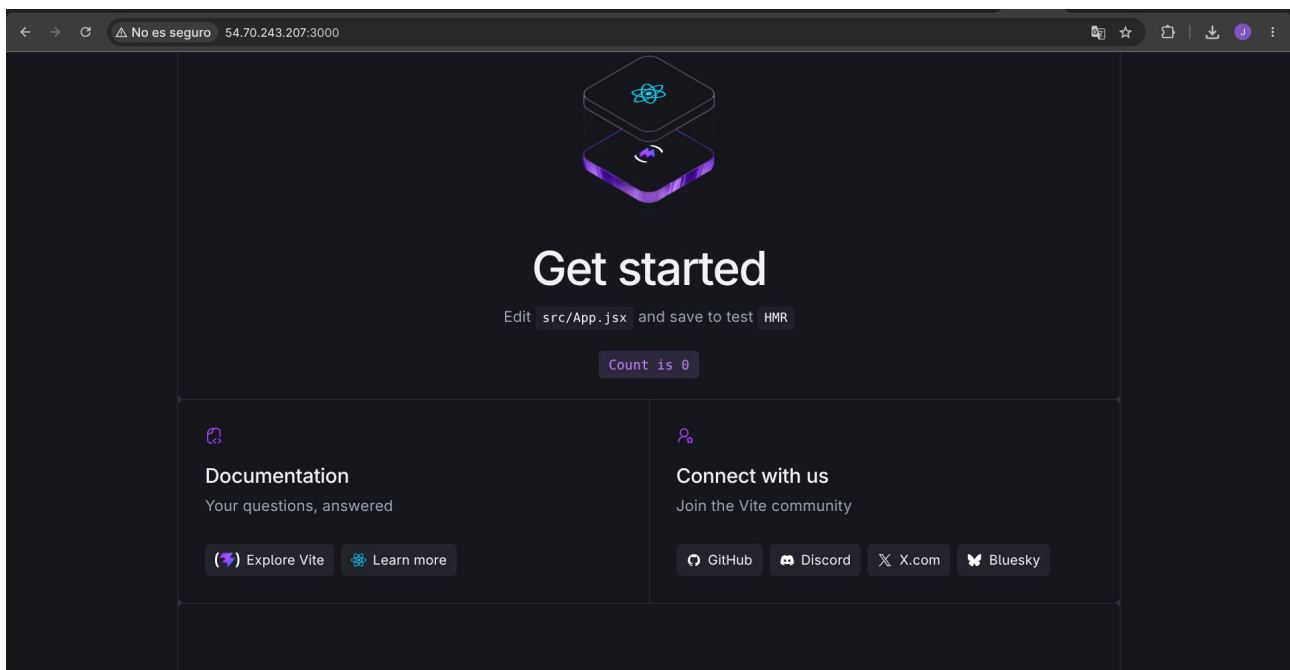


Figura 11: Frontend de SantaMed funcionando desde la IP pública del servidor. Por ahora muestra la estructura base; en el próximo avance se le suman las pantallas que consumen la API de profesionales.

5. Próximos pasos

Desarrollar las pantallas del frontend que consumen la API (listado de profesionales con filtros y formularios de alta y edición), a cargo del equipo de Frontend & QA, y configurar el balanceador HAProxy.